

Final

STAT 651

RJ Cass

2026-04-18

```
suppressPackageStartupMessages(library('coda'))
suppressPackageStartupMessages(library('knitr'))
suppressPackageStartupMessages(library('invgamma'))
suppressPackageStartupMessages(library('rstan'))
set.seed(1337)
```

```
run_diagnostics <- function(samples_list) {
  mcmc_obj <- coda::mcmc.list(
    lapply(samples_list, coda::mcmc)
  )

  raftery_results <- lapply(mcmc_obj, coda::raftery.diag)
  names(raftery_results) <- paste0("Chain_", 1:length(raftery_results))

  list(
    Samples = mcmc_obj,
    Gelman = coda::gelman.diag(mcmc_obj),
    Raftery = raftery_results,
    ESS = coda::effectiveSize(mcmc_obj)
  )
}
```

```
run_diagnostics_stan <- function(fit_stan) {
  mcmc_obj <- As.mcmc.list(fit_stan)

  raftery_results <- lapply(mcmc_obj, coda::raftery.diag)
  names(raftery_results) <- paste0("Chain_", 1:length(raftery_results))

  list(
    Samples = mcmc_obj,
    Gelman = coda::gelman.diag(mcmc_obj),
    Raftery = raftery_results,
    ESS = coda::effectiveSize(mcmc_obj)
  )
}
```

```

height <- c(10, 14, 17, 21, 24, 28, 31, 35, 39, 42, 46, 49, 53, 56, 60)
distance <- c(174.3, 160.1, 162.8, 174.6, 172.6, 184.4, 181.8, 185.0, 192.4, 193.3, 180.6, 172.

n <- length(height)

beta_means_prior <- c(200, 0, 0)
beta_var_prior <- c(75^2, 50^2, 50^2)
error_var_prior_n0 <- 5
error_var_prior_s02 <- 5

error_var_prior_shape <- error_var_prior_n0/2
error_var_prior_scale <- error_var_prior_n0 * error_var_prior_s02/2

height2 <- height*height

```

1

(a)

```

X <- model.matrix(distance ~ height + height2)

lm_bayes <- function(y, X, iters = 5000, burnin = 100,
  inits = list(
    beta = rep(0, ncol(X)),
    gamma = 1/100
  ),
  priors = list(
    # Prior params for betas
    beta_0 = rep(0, ncol(X)),
    Sigma_0_inv = solve(100000*diag(ncol(X))),
    # Prior params for sigma^2
    nu_0 = 0.01,
    sigma2_0 = 1
  )
) {

  n <- nrow(X)
  p <- ncol(X)

  samps <- matrix(nrow = iters, ncol = p+1)

  gamma <- inits$gamma
  beta <- inits$beta

```

```

# Precompute some stuff that will be used in the loop
XtX <- crossprod(X)
Xty <- crossprod(X, y)
Xty_prior <- crossprod(priors$Sigma_0_inv, priors$beta_0)
shape_prior <- (priors$nu_0 + n)/2
scale_prior <- priors$nu_0 * priors$sigma2_0

for (i in 1:(iters+burnin)) {
  # Update beta | gamma, data
  A <- gamma*XtX + priors$Sigma_0_inv
  A_inv <- solve(A)
  B <- gamma*Xty + Xty_prior
  beta <- MASS::mvrnorm(n=1, A_inv%*%B, A_inv)

  # Update gamma | beta, data
  SSR <- crossprod(y - X%*%beta)
  gamma <- rgamma(1, shape = shape_prior, scale = .5*(SSR + scale_prior))

  if (i > burnin){
    samp[samp[i-burnin, ] <- c(beta, 1/gamma)
  }
}
samps
}

priors <- list(beta_0 = beta_means_prior, Sigma_0_inv = solve(100000*diag(ncol(X))), nu_0 = er
sigma2_0 = error_var_prior_s02)

# To run on multiple chains
samples <- vector("list", 3)
for (chain in 1:length(samples)) {
  set.seed(1337+chain)
  beta_start <- MASS::mvrnorm(n=1, mu = rep(0, 7), Sigma = 10000*diag(7))
  gamma_start <- rgamma(1, 5, 10)
  samples[[chain]] <- lm_bayes(y = distance, X, inits = list(beta = beta_start, gamma = gamma_s
}

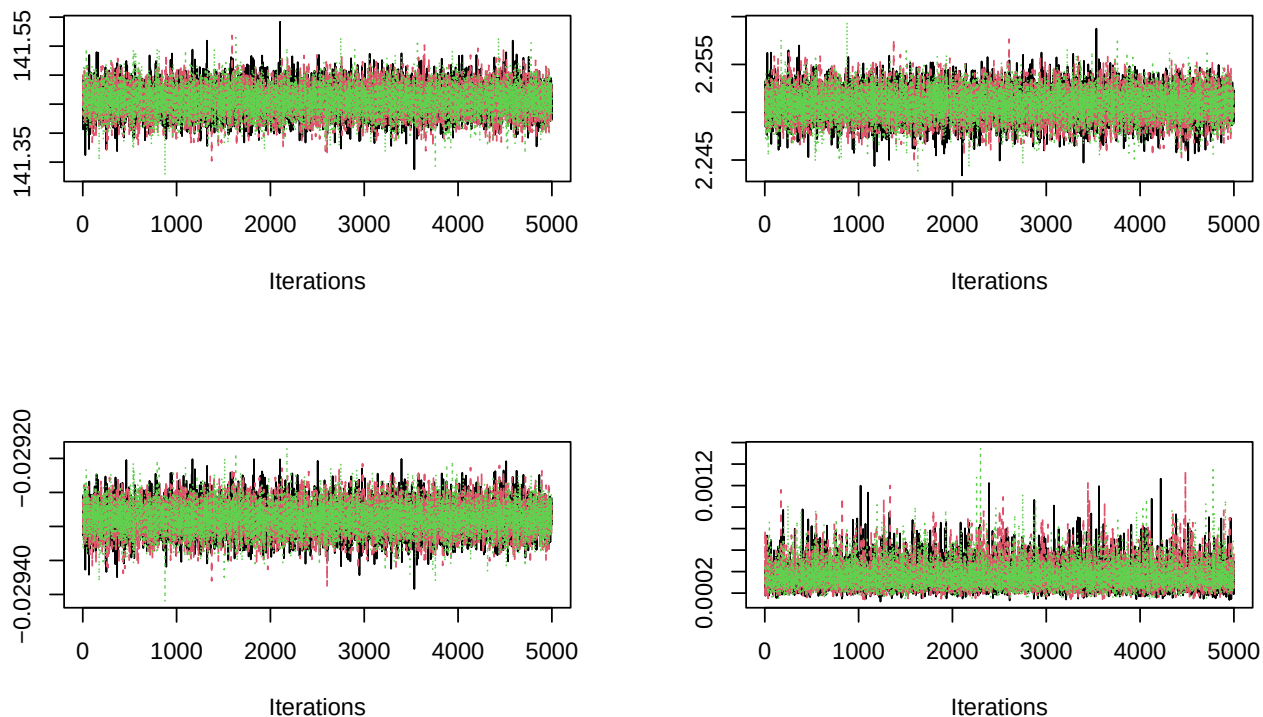
samples_diag <- run_diagnostics(samples)
all_samps_1 <- as.matrix(samples_diag$Samples)
beta0 <- all_samps_1[, 1]
beta1 <- all_samps_1[, 2]
beta2 <- all_samps_1[, 3]
sigma2 <- all_samps_1[, 4]
post_y <- colMeans(sapply(height, \ (x) beta0 + beta1*x + beta2 * x^2 + sigma2))

```

(b)

The first thing I will explore are the trace plots of the chains:

```
par(mfrow = c(2, 2))
coda::traceplot(samples_diag$Samples)
```



```
par(mfrow = c(1,1))
```

As can be seen from the traceplots, each chain appears to converge to a similar value for each parameter.

We have the following Effective Sample Sizes for each parameter:

```
samples_diag$ESS
```

var1	var2	var3	var4
13782.50	13606.36	14482.46	15289.03

As is seen, we have well over 3000 ESS per parameter.

Looking at our Gelman values ($\hat{R}$), which compares the variance within a chain to the variance across all chains, we get the following:

```
samples_diag$Gelman
```

Potential scale reduction factors:

	Point est.	Upper C.I.
[1,]	1	1
[2,]	1	1
[3,]	1	1
[4,]	1	1

Multivariate psrf

1

These values are all fine (a troublesome $\hat{R}$ is usually above 1.1) meaning the variance between chains and within chains is relatively similar (we don't have one chain acting wildly differently from the others)

Examining the Raftery values, we have the following:

```
samples_diag$Raftery
```

\$Chain_1

Quantile (q) = 0.025
Accuracy (r) = +/- 0.005
Probability (s) = 0.95

Burn-in (M)	Total (N)	Lower bound (Nmin)	Dependence factor (I)
2	3866	3746	1.030
2	3680	3746	0.982
2	3930	3746	1.050
2	3930	3746	1.050

\$Chain_2

Quantile (q) = 0.025
Accuracy (r) = +/- 0.005
Probability (s) = 0.95

Burn-in (M)	Total (N)	Lower bound (Nmin)	Dependence factor (I)
2	3930	3746	1.050
2	3680	3746	0.982

2	3680	3746	0.982
2	3680	3746	0.982

\$Chain_3

Quantile (q) = 0.025
 Accuracy (r) = +/- 0.005
 Probability (s) = 0.95

Burn-in (M)	Total (N)	Lower bound (Nmin)	Dependence factor (I)
2	3803	3746	1.020
2	3620	3746	0.966
2	3741	3746	0.999
2	3620	3746	0.966

In these tables, the ‘Dependence Factor’ is the important number, with values around 1 (ie. not high) indicating that we have run our chains long enough to get resonable estimates of parameter values.

(c)

The estimates of posterior means and 95% confidence intervals is shown in Table 1.

```
Estimate <- round(colMeans(all_samps_1), 3)
Lower <- round(apply(all_samps_1, 2, quantile, probs = c(.025)), 3)
Upper <- round(apply(all_samps_1, 2, quantile, probs = c(.975)), 3)
Variable <- c('beta0', 'beta1', 'beta2', 'sigma2')
post_est <- as.data.frame(cbind(Variable, Estimate, Lower, Upper))
rownames(post_est) <- NULL
kable(post_est)
```

Table 1: MonteCarlo estimates of posterior means for each parameter, with 95% equal-tailed intervals

Variable	Estimate	Lower	Upper
beta0	141.457	141.406	141.508
beta1	2.251	2.248	2.254
beta2	-0.029	-0.029	-0.029
sigma2	0	0	0.001

(d)

```
phi <- -1 * beta1 / (2*beta2)

phi_est <- round(mean(phi), 2)
phi_lower <- round(quantile(phi, probs = c(.025)), 2)
phi_upper <- round(quantile(phi, probs = c(.975)), 2)
```

For the value of $\phi = \frac{-\beta_1}{2\beta_2}$ we have a posterior mean of 38.43, with a 95% interval from 38.41 to 38.44.

(e)

The distribution of the data, with the posterior mean estimate of ϕ (vertical red line) and the posterior mean estimate of the y 's, is provided in Figure 1. As is shown, the posterior estimate of the y 's is much higher than the actual distribution, implying the model is at least a decent fit for the data. However, the posterior ϕ (the optimum height) does appear to be off a little bit.

```
plot(height, distance)
abline(h = mean(post_y), col = 'blue')
abline(v = phi_est, col = 'red')
```

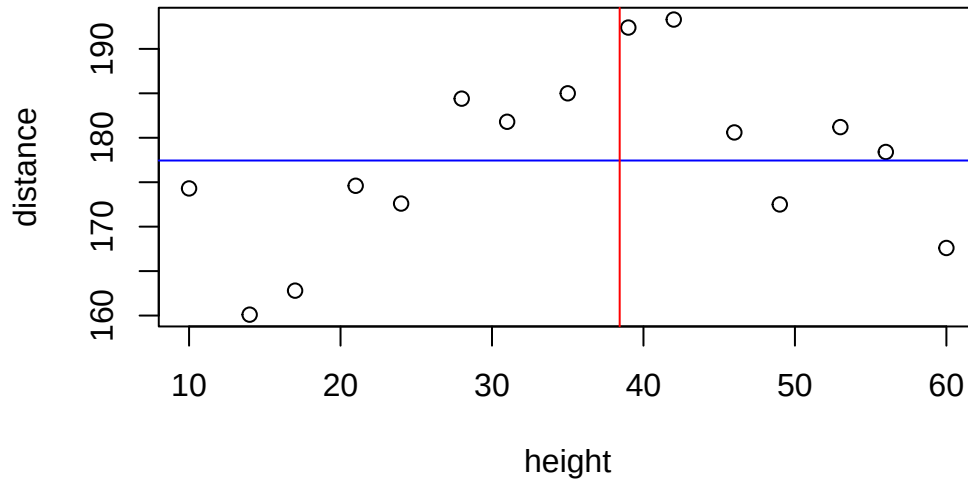


Figure 1: Original data with posterior estimates of Phi and the y 's

2

(a)

We now have $y_i = \alpha - \beta(x_i - \phi)^2 + \epsilon_i$. To show that this is the same as the original likelihood:

$$y_i = \alpha - \beta(x_i^2 - 2\phi x_i + \phi^2) + \epsilon_i = \alpha - \beta x_i^2 + 2\beta\phi x_i - \beta\phi^2 + \epsilon_i$$

If we let

$$\begin{aligned}\beta_0 &= \alpha - \beta\phi^2 \\ \beta_1 &= 2\beta\phi \\ \beta_2 &= -\beta \\ \epsilon_i &= \epsilon_i\end{aligned}$$

We can rewrite this as:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \epsilon_i$$

(b)

$$p(y|x, \alpha, \beta, \phi, \sigma^2) \sim N(\alpha - \beta(x - \phi)^2, \epsilon) \propto (\epsilon)^{-n/2} e^{-\frac{1}{2\epsilon} \sum (x_i - (\alpha - \beta(x_i - \phi)^2))}$$

$$p(\alpha) \propto e^{-\frac{1}{2(50^2)}(\alpha - 220)^2}$$

$$p(\beta) \propto \beta^{2-1} e^{-10\beta}$$

$$p(\phi) = e^{-\frac{1}{2(3^2)}(\phi - 30)^2}$$

$$p(\sigma^2) = (\sigma^2)^{-5/2-1} e^{-\frac{12.5}{(\sigma^2)}}$$

$$p(y, x, \alpha, \beta, \phi, \sigma^2) = p(y|x, \alpha, \beta, \phi, \sigma^2)p(\alpha)p(\beta)p(\phi)p(\sigma^2)$$

$$p(\alpha|y, x, \beta, \phi, \sigma^2) \propto p(y|x, \alpha, \beta, \phi, \sigma^2)p(\alpha)$$

$$p(\beta|y, x, \alpha, \phi, \sigma^2) \propto p(y|x, \alpha, \beta, \phi, \sigma^2)p(\beta)$$

$$p(\phi|y, x, \alpha, \beta, \sigma^2) \propto p(y|x, \alpha, \beta, \phi, \sigma^2)p(\phi)$$

$$p(\sigma^2|y, x, \alpha, \beta, \phi) \propto p(y|x, \alpha, \beta, \phi, \sigma^2)p(\sigma^2)$$

To skip over the math, I'll just do an MH sampler for EVERY variable. Either that or a PPL.

(c)

```
alpha_prior_mean <- 220
alpha_prior_var <- 50^2
beta_prior_shape <- 2
beta_prior_rate <- 10
phi_prior_mean <- 30
phi_prior_var <- 3^2
sigma2_prior_shape <- 5/2
sigma2_prior_scale <- 5*5/2
```

```

loglik_y <- function(X, alpha, beta, phi, epsilon, log_prior) {
  log_lik <- sum(alpha - beta*(X - phi)^2 + epsilon)
  log_lik + log_prior
}

mh_step <- function(X, alpha, beta, phi, epsilon, var, proposal, log_prior_curr, log_prior_prop) {
  log_c <- loglik_y(X, alpha, beta, phi, epsilon, log_prior_curr)
  if (var == 'alpha') {
    curr <- alpha
    log_p <- loglik_y(X, proposal, beta, phi, epsilon, log_prior_prop)
  } else if (var == 'beta') {
    curr <- beta
    log_p <- loglik_y(X, alpha, proposal, phi, epsilon, log_prior_prop)
  } else if (var == 'phi') {
    curr <- phi
    log_p <- loglik_y(X, alpha, beta, proposal, epsilon, log_prior_prop)
  } else {
    curr <- epsilon
    log_p <- loglik_y(X, alpha, beta, phi, proposal, log_prior_prop)
  }

  diff <- log_p - log_c

  if (log(runif(1)) < diff) {
    # Accept
    return(c(res = proposal, acc = 1))
  } else {
    # Reject
    return(c(res = curr, acc = 0))
  }
}

```

```

lm_bayes_2 <- function(y, X, iters = 5000, burnin = 100,
  inits = list(
    alpha = 0,
    beta = 0,
    phi = 0,
    epsilon = 0
  )
){
  n <- nrow(X)
  p <- ncol(X)

  samps <- matrix(nrow = iters, ncol = 4)

  alpha <- inits$alpha

```

```

beta <- inits$beta
phi <- inits$phi
epsilon <- inits$epsilon

acc_alpha <- 0
acc_beta <- 0
acc_phi <- 0
acc_epsilon <- 0

for (i in 1:(iters+burnin)) {
  # Update alpha
  alpha_prop <- rnorm(1, mean = alpha_prior_mean, sd = sqrt(alpha_prior_var))
  alpha_log_prior_curr <- dnorm(alpha, mean = alpha_prior_mean, sd = sqrt(alpha_prior_var), log = TRUE)
  alpha_log_prior_prop <- dnorm(alpha_prop, mean = alpha_prior_mean, sd = sqrt(alpha_prior_var), log = TRUE)
  step <- mh_step(X, alpha, beta, phi, epsilon, var = 'alpha', alpha_prop, alpha_log_prior_curr, alpha_log_prior_prop)
  alpha <- step[[1]]
  acc_alpha <- acc_alpha + step[[2]]
  # Update beta
  beta_prop <- rgamma(1, shape = beta_prior_shape, rate = beta_prior_rate)
  beta_log_prior_curr <- dgamma(beta, shape = beta_prior_shape, rate = beta_prior_rate, log = TRUE)
  beta_log_prior_prop <- dgamma(beta_prop, shape = beta_prior_shape, rate = beta_prior_rate, log = TRUE)
  step <- mh_step(X, alpha, beta, phi, epsilon, var = 'beta', beta_prop, beta_log_prior_curr, beta_log_prior_prop)
  beta <- step[[1]]
  acc_beta <- acc_beta + step[[2]]
  # Update phi
  phi_prop <- rnorm(1, mean = phi_prior_mean, sd = sqrt(phi_prior_var))
  phi_log_prior_curr <- dnorm(phi, mean = phi_prior_mean, sd = sqrt(phi_prior_var), log = TRUE)
  phi_log_prior_prop <- dnorm(phi_prop, mean = phi_prior_mean, sd = sqrt(phi_prior_var), log = TRUE)
  step <- mh_step(X, alpha, beta, phi, epsilon, var = 'phi', phi_prop, phi_log_prior_curr, phi_log_prior_prop)
  phi <- step[[1]]
  acc_phi <- acc_phi + step[[2]]
  # Update epsilon
  epsilon_prop <- rinvgamma(1, shape = sigma2_prior_shape, scale = sigma2_prior_scale)
  epsilon_log_prior_curr <- dinvgamma(epsilon, shape = sigma2_prior_shape, scale = sigma2_prior_scale, log = TRUE)
  epsilon_log_prior_prop <- dinvgamma(epsilon_prop, shape = sigma2_prior_shape, scale = sigma2_prior_scale, log = TRUE)
  step <- mh_step(X, alpha, beta, phi, epsilon, var = 'epsilon', epsilon_prop, epsilon_log_prior_curr, epsilon_log_prior_prop)
  epsilon <- step[[1]]
  acc_epsilon <- acc_epsilon + step[[2]]

  if (i > burnin) {
    sampsi[i - burnin, ] <- c(alpha, beta, phi, epsilon)
  }
}

#cat("Acceptance Rate (alpha):", acc_alpha / iters, "\n")
#cat("Acceptance Rate (beta):", acc_beta / iters, "\n")
#cat("Acceptance Rate (phi):", acc_phi / iters, "\n")

```

```

#cat("Acceptance Rate (epsilon):", acc_epsilon / iters, "\n")
return(samps)
}

samples_2 <- vector("list", 3)
for (s in 1:length(samples_2)) {

  alpha_start <- rnorm(1, mean = alpha_prior_mean, sd = sqrt(alpha_prior_var))
  beta_start <- rgamma(1, shape = beta_prior_shape, rate = beta_prior_rate)
  phi_start <- rnorm(1, mean = phi_prior_mean, sd = sqrt(phi_prior_var))
  epsilon_start <- rinvgamma(1, shape = sigma2_prior_shape, scale = sigma2_prior_scale)

  samples_2[[s]] <- lm_bayes_2(y = height, X = distance, iters = 10000, burnin = 100,
    inits = list(alpha = alpha_start, beta = beta_start, phi = phi_start, epsilon = epsilon_start)
  )
}

samples_diag <- run_diagnostics(samples_2)
all_samps_2 <- as.matrix(samples_diag$Samples)
alpha <- all_samps_2[, 1]
beta <- all_samps_2[, 2]
phi <- all_samps_2[, 3]
epsilon <- all_samps_2[, 4]
post_y <- colMeans(sapply(height, \(x) alpha - beta*(x - phi)^2 + epsilon))

samples_diag$ESS

```

```

      var1      var2      var3      var4
45.93503 38.61547 23.12942 16.19919

```

OK, clearly something is off with my proposals or the likelihood or something, so I will pivot to a PPL.

```

X <- model.matrix(distance ~ height + height2)

data_stan <- list(N = length(height), y = height, x = X,
  alpha_mean = alpha_prior_mean, alpha_sd = sqrt(alpha_prior_var),
  beta_shape = beta_prior_shape, beta_rate = beta_prior_rate,
  phi_mean = phi_prior_mean, phi_sd = sqrt(phi_prior_var),
  sig2_shape = sigma2_prior_shape, sig2_scale = sigma2_prior_scale)

params <- c('alpha', 'beta', 'phi', 'epsilon')

fit_stan <- stan(model_code = readLines("q2.stan"),
  data = data_stan, pars = params,
  iter = 10000, warmup = 500, chains = 3)

```

```

mcmc_obj <- As.mcmc.list(fit_stan)

samples_diag_2 <- run_diagnostics_stan(fit_stan)
all_samps_2 <- as.matrix(samples_diag_2$Samples)
all_samps_2 <- all_samps_2[, -5]
alpha <- all_samps_2[, 1]
beta <- all_samps_2[, 2]
phi <- all_samps_2[, 3]
epsilon <- all_samps_2[, 4]
post_y <- colMeans(sapply(height, \(x) alpha - beta*(x - phi)^2 + epsilon))

```

STAN Code

```

data {
  int<lower=0> N;
  array[N] real y;
  matrix[N,3] x;
  real alpha_mean;
  real alpha_sd;
  real beta_shape;
  real beta_rate;
  real phi_mean;
  real phi_sd;
  real sig2_shape;
  real sig2_scale;
}

parameters {
  real alpha;
  real beta;
  real phi;
  real<lower=0> epsilon;
}

transformed parameters {
  vector[3] betas;
  betas[1] = alpha - beta*phi^2; // Multiply by tau
  betas[2] = 2 * beta * phi;
  betas[3] = -1 * beta;
}

model {
  alpha ~ normal(alpha_mean, alpha_sd);
  beta ~ gamma(beta_shape, beta_rate);
  phi ~ normal(phi_mean, phi_sd);
}

```

```

epsilon ~ inv_gamma(sig2_shape, sig2_scale);

y ~ normal(x * betas, epsilon);
}

```

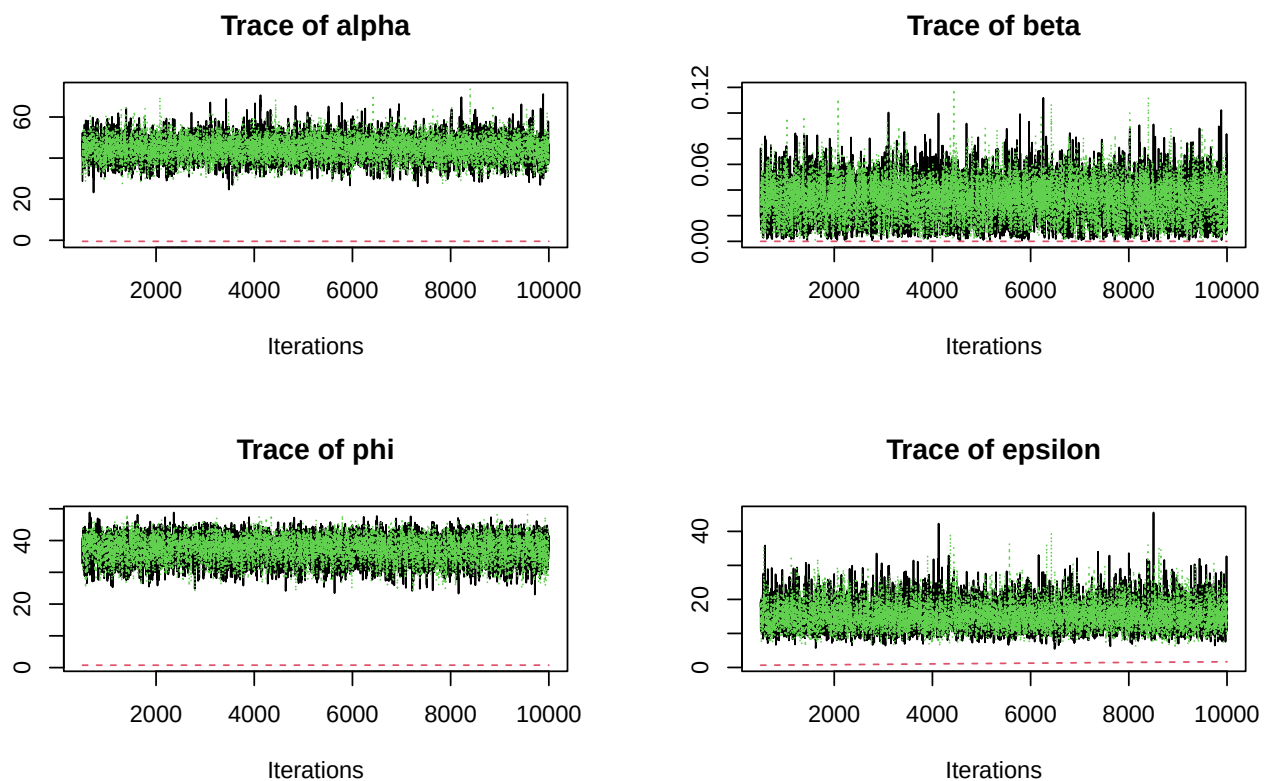
(d)

The first thing I will explore are the trace plots of the chains:

```

par(mfrow = c(2, 2))
coda::traceplot(samples_diag_2$Samples[, -5])

```



```

par(mfrow = c(1,1))

```

As can be seen from the traceplots, each chain appears to converge to a similar value for each parameter.

We have the following Effective Sample Sizes for each parameter:

```

samples_diag_2$ESS

```

```

alpha    beta    phi    epsilon    lp__
4998.840 4592.211 3736.213 5355.157 6061.837

```

As is seen, we have well over 3000 ESS per parameter.

Looking at our Gelman values ($\hat{R}$), which compares the variance within a chain to the variance across all chains, we get the following:

```
samples_diag_2$Gelman
```

Potential scale reduction factors:

	Point est.	Upper C.I.
alpha	8.20	19.97
beta	2.27	4.92
phi	9.34	22.82
epsilon	3.60	8.40
lp__	6.99	42.91

Multivariate psrf

14.2

We have some very poor values here, with values being HUGE. This indicates perhaps poor mixing (requiring perhaps reparameterization, or poor tuning)

Examining the Raftery values, we have the following:

```
samples_diag_2$Raftery
```

\$Chain_1

Quantile (q) = 0.025

Accuracy (r) = +/- 0.005

Probability (s) = 0.95

	Burn-in (M)	Total (N)	Lower bound (Nmin)	Dependence factor (I)
alpha	8	9087	3746	2.43
beta	10	11730	3746	3.13
phi	8	8877	3746	2.37
epsilon	15	17127	3746	4.57
lp__	6	6440	3746	1.72

\$Chain_2

Quantile (q) = 0.025

Accuracy (r) = +/- 0.005

Probability (s) = 0.95

	Burn-in (M)	Total (N)	Lower bound (Nmin)	Dependence factor (I)
alpha	126	124994	3746	33.40
beta	5	5491	3746	1.47
phi	154	166854	3746	44.50
epsilon	535	394942	3746	105.00
lp__	807	451142	3746	120.00

\$Chain_3

Quantile (q) = 0.025
Accuracy (r) = +/- 0.005
Probability (s) = 0.95

	Burn-in (M)	Total (N)	Lower bound (Nmin)	Dependence factor (I)
alpha	10	11820	3746	3.16
beta	7	8024	3746	2.14
phi	12	14622	3746	3.90
epsilon	12	13214	3746	3.53
lp__	5	5983	3746	1.60

In this case, some of our chains are ok-sh, but Chain 2 in particular is ROUGH (huge I values), indicating we likely won't have good prediction from it.

(e)

The estimates of posterior means and 95% confidence intervals is shown in Table 1.

```
Estimate <- round(colMeans(all_samps_2), 4)
Lower <- round(apply(all_samps_2, 2, quantile, probs = c(.025)), 4)
Upper <- round(apply(all_samps_2, 2, quantile, probs = c(.975)), 4)
Variable <- c('alpha', 'beta', 'phi', 'epsilon')
post_est <- as.data.frame(cbind(Variable, Estimate, Lower, Upper))
rownames(post_est) <- NULL
kable(post_est)
```

Table 2: MonteCarlo estimates of posterior means for each parameter, with 95% equal-tailed intervals

Variable	Estimate	Lower	Upper
alpha	29.5639	-0.5281	54.7887
beta	0.0219	0	0.0616
phi	25.206	0.7597	43.8726
epsilon	10.3466	0.7242	23.1129

Table 2: *MonteCarlo estimates of posterior means for each parameter, with 95% equal-tailed intervals*

Variable	Estimate	Lower	Upper
----------	----------	-------	-------

(f)

MY POSTERIOR Ys ARE WRONG SO THESE NEXT 2 SECTIONS ARE ALSO JUST WRONG.
I didn't have time to figure out what was going wrong.

For each simulated parameter set, generate a random value for y. Then, do that a bunch of times and find the number that are over 187.

```
prob_187 <- sapply(35, \(x) alpha - beta*(x - phi)^2 + epsilon)
```

(g)

Similar to above, but you only do it once to simulate only the next drive.

```
# For all simulated parameter values, draw a corresponding prediction for a year
post_preds <- mean(sapply(35, \(x) alpha - beta*(x - phi)^2 + epsilon))
```